# JOBSHEET 04

# ROUTING & LAYOUTING PADA NEXT.JS

# PEMOGRAMAN BERBASIS FRAMEWORK

**REIKA AMALIA SYAHPUTRI**

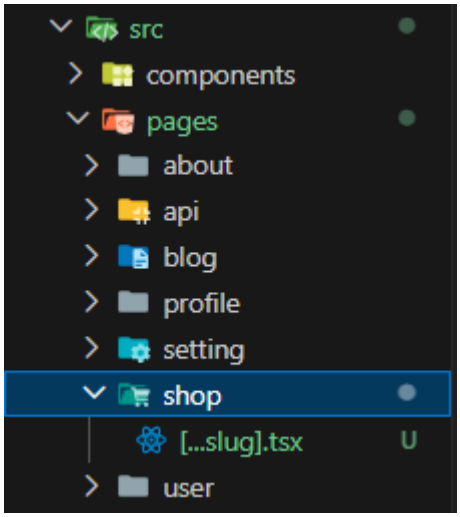
**2341720173**

**TI-3E / 19**

**TEKNOLOGI INFORMASI**

## Langkah 2 – Membuat Catch-All Route

1. Masuk ke folder pages.
2. Buat folder shop dan file […slug].tsx:

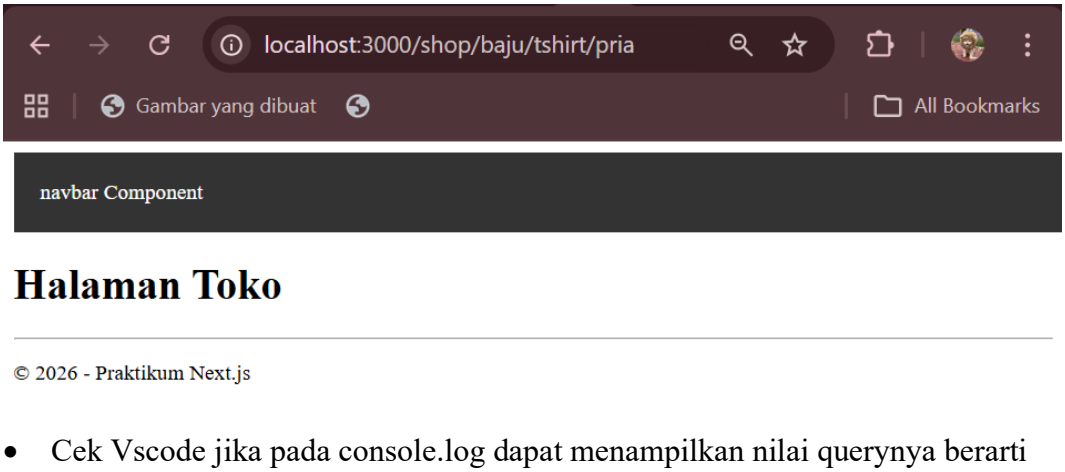



3. Modifikasi Isi file […slug].tsx dengan kode berikut:

```
import { useRouter } from "next/router";

const halamanToko = () => {
    const Router = useRouter();
    console.log(Router);
    // const { query } = useRouter();
return (
<div>
    <h1>Halaman Toko</h1>

</div>
    );
};

export default halamanToko;
```

Cek menggunakan console.log apakah nilai segment berhasil didapat

- Jalankan browser dan ketik urlnya sebagai berikut




- Cek Vscode jika pada console.log dapat menampilkan nilai querynya berarti berhasil

```
my-app-02 > src > pages > shop > [...slug].tsx > halamanToko
import { useRouter } from "next/router";

Windsurf: Refactor | Explain | Generate JSDoc | ×
const halamanToko = () => {
    const Router = useRouter();
    console.log(Router);  ×5 ᵇ { pathname: '/shop/[...slug]', route: '/shop/[...slug]', query: { slug: [...] }, asPath: '/shop/baju/tshirt/pria', components:...
    // const { query } = useRouter();
return (
<div>
    <h1>Halaman Toko</h1>

</div>
    );
};

export default halamanToko;
```

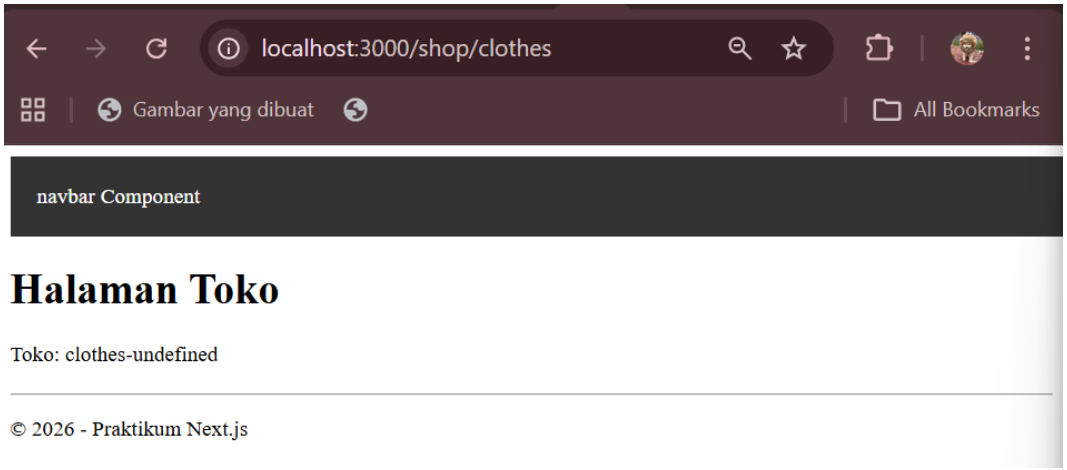
- Modifikasi [...slug].tsx untuk menampilkan nilai query

```
import { useRouter } from "next/router";

const halamanToko = () => {
    // const Router = useRouter();
    // console.log(Router);
    const { query } = useRouter();
return (
<div>
    <h1>Halaman Toko</h1>
    <p>Toko: {`${query.slug && query.slug[0]+"-"+ query.slug[1]}`}</p> {/* menggunakan backtick bukan petik satu*/}

</div>
    );
};

export default halamanToko;
```

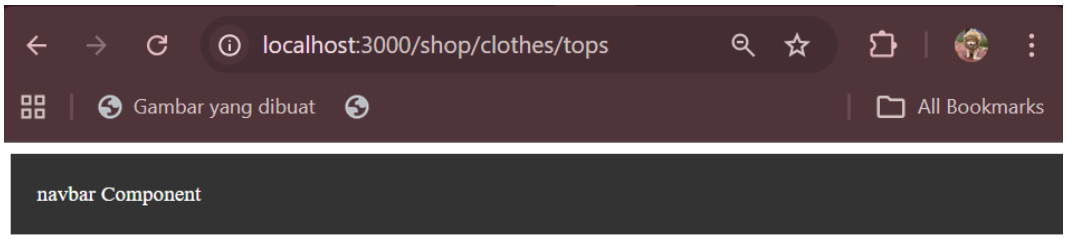
## Langkah 3 – Pengujian Catch-All Route

Akses URL berikut di browser:

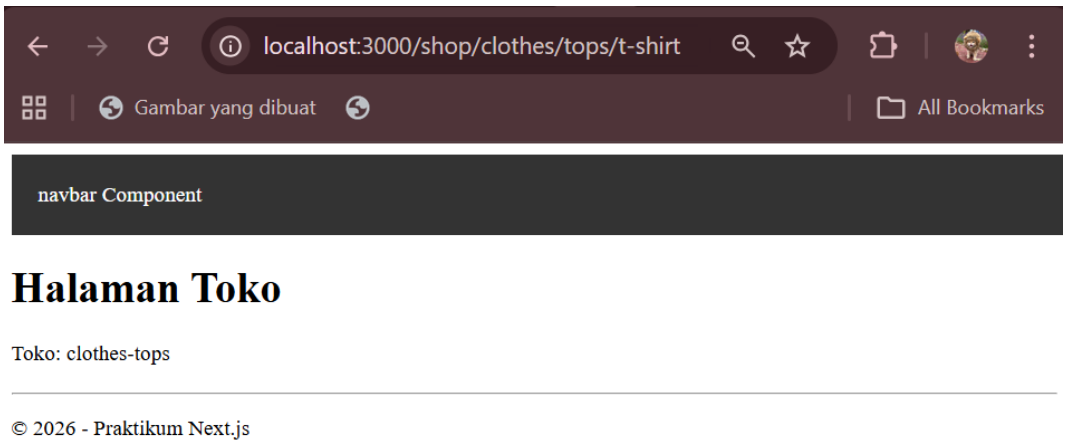
/shop/clothes




/shop/clothes/tops




/shop/clothes/tops/t-shirt

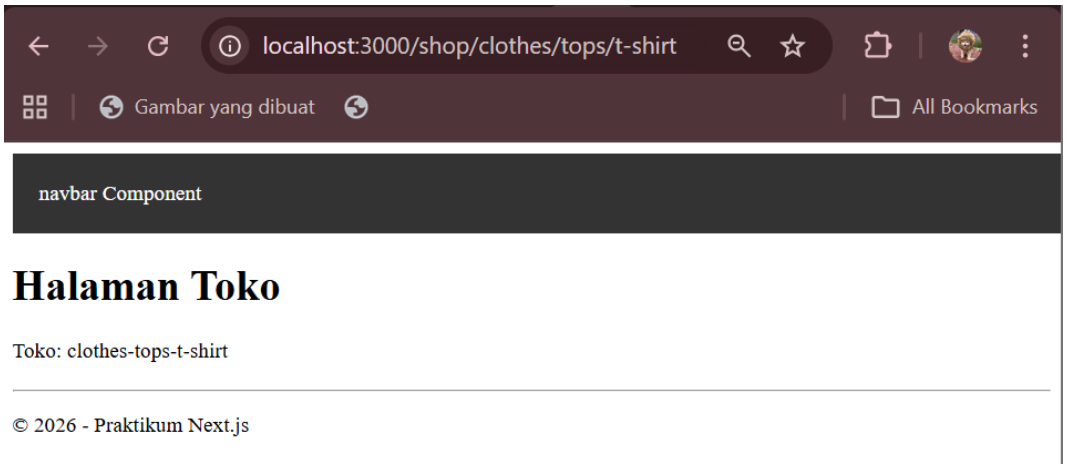


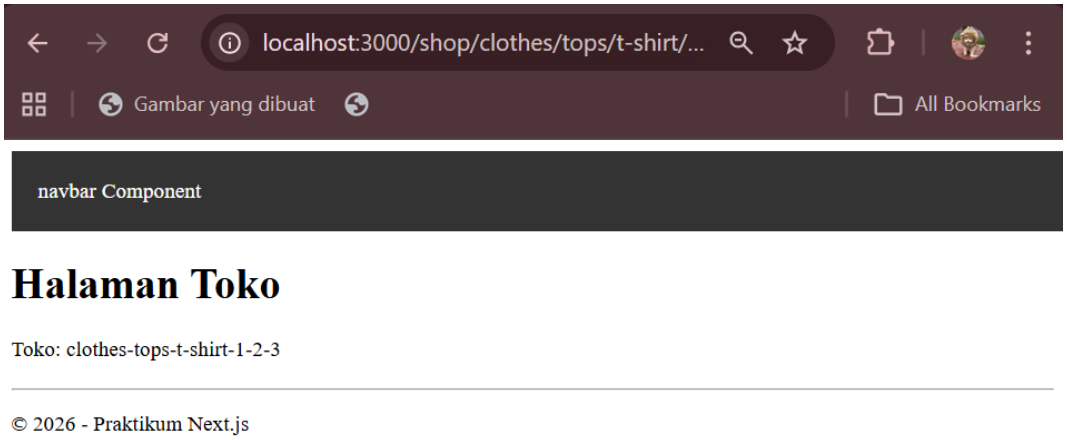

Jika dilihat ada yang terbaca undifined dan ada yang tidak terbaca ini dikarena segmennya dibatasi Cuma array[0] dan array[1]. Solusinya bagaimana ?

Modifikasi […slug].tsx menjadi berikut

```
import { useRouter } from "next/router";

const halamanToko = () => {
    // const Router = useRouter();
    // console.log(Router);
    const { query } = useRouter();
return (
<div>
    <h1>Halaman Toko</h1>
    {/* <p>Toko: {`${query.slug && query.slug[0]+"-"+ query.slug[1]}`}</p> menggunakan backtick bukan petik satu */}
    <p>Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}</p>  {/* Berapapun banyaknya seqment tetap terbaca  */}
</div>
    );
};

export default halamanToko;
```

Jalankan browser : Berapapun banyaknya seqment tetap terbaca




Untuk saat ini gunakan

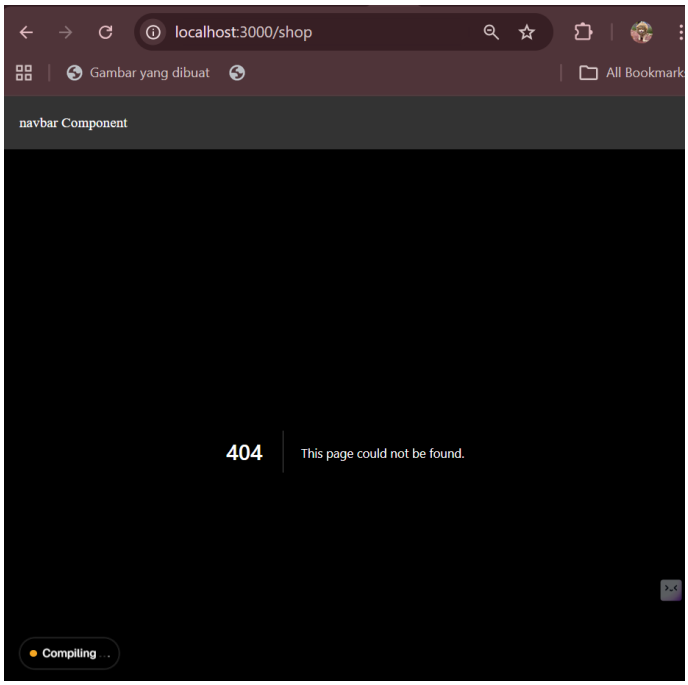
```
import { useRouter } from "next/router";

const halamanToko = () => {
    // const Router = useRouter();
    // console.log(Router);
    const { query } = useRouter();
return (
<div>
    <h1>Halaman Toko</h1>
    <p>Toko: {`${query.slug && query.slug[0]+"-"+ query.slug[1]}`}</p>  {/* menggunakan backtick bukan petik satu */}
    {/* <p>Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}</p>  Berapapun banyaknya seqment tetap terbaca  */}
</div>
    );
};

export default halamanToko;
```

Perhatikan bahwa:

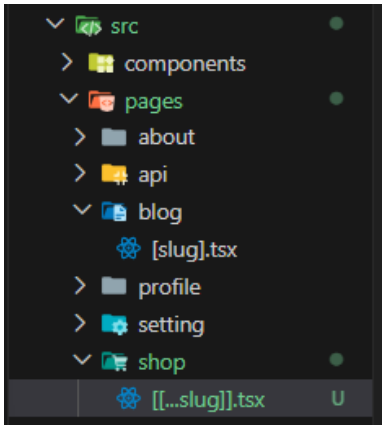
- slug berbentuk array
- Isi halaman berubah sesuai URL

# Langkah 4 – Optional Catch-All Route

1. Jika menggunakan [...slug].js maka ketika mengakses shop akan terjadi error

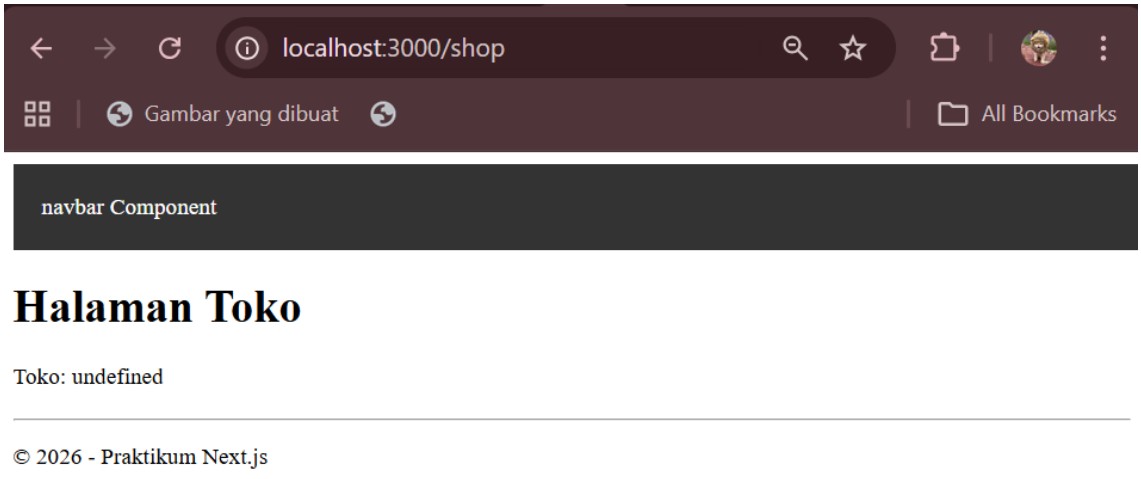



2. Solusinya dengan Rename file: [...slug].tsx → [[...slug]].tsx




3. Sekarang akses:

/shop




4. Halaman dapat diakses meskipun tanpa parameter

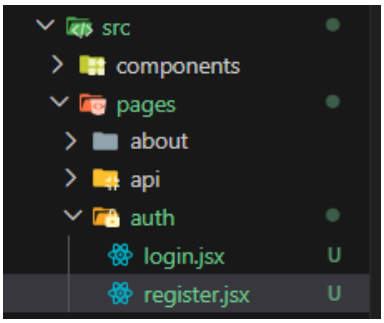
## Langkah 5 – Validasi Parameter

Tambahkan validasi agar tidak error saat slug kosong:

```jsx
import { useRouter } from "next/router";

const halamanToko = () => {
    // const Router = useRouter();
    // console.log(Router);
    const { query } = useRouter();
    const slug = query.slug as string[] | undefined;

return (
<div>
    <h1>Halaman Toko</h1>
    <p>Kategori: {slug ? slug[0] : "Semua Kategori"}</p>
     <p>
        Toko: {Array.isArray(slug) ? slug.join("-") : "-"}
      </p>
    {/* <p>Toko: {`${query.slug && query.slug[0]+"-"+ query.slug[1]}`}</p>  menggunakan backtick bukan petik satu */}
    {/* <p>Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}</p>  Berapapun banyaknya seqment tetap terbaca  */}
</div>
    );
};

export default halamanToko;
```

# Langkah 6 – Membuat Halaman Login & Register

1. Buat folder:
2. pages/auth
3. Buat file:
   - login.jsx
   - register.jsx




4. Modifikasi login.jsx:

```jsx
import Link from "next/link";

const halamanLogin = () => {
    return (
        <div>
            <h1>Halaman Login</h1>
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default halamanLogin;
```

5. Modifikasi register.jsx

```jsx
import Link from "next/link";

const halamanRegister = () => {
    return (
        <div>
            <h1>Halaman Register</h1>
            <Link href="/auth/login">Ke Halaman Login</Link>
        </div>
    );
}

export default halamanRegister;
```

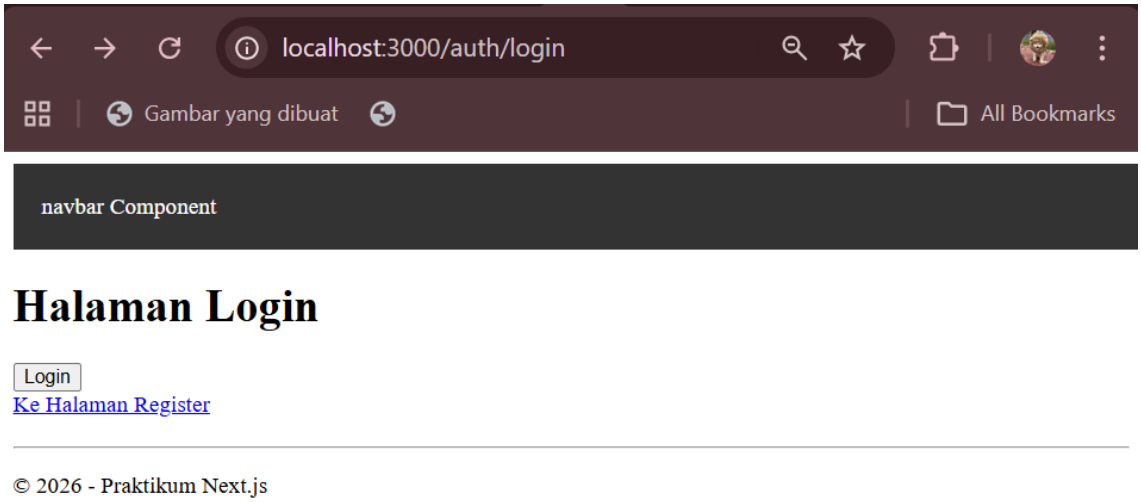
## Langkah 7 – Navigasi Imperatif (router.push)

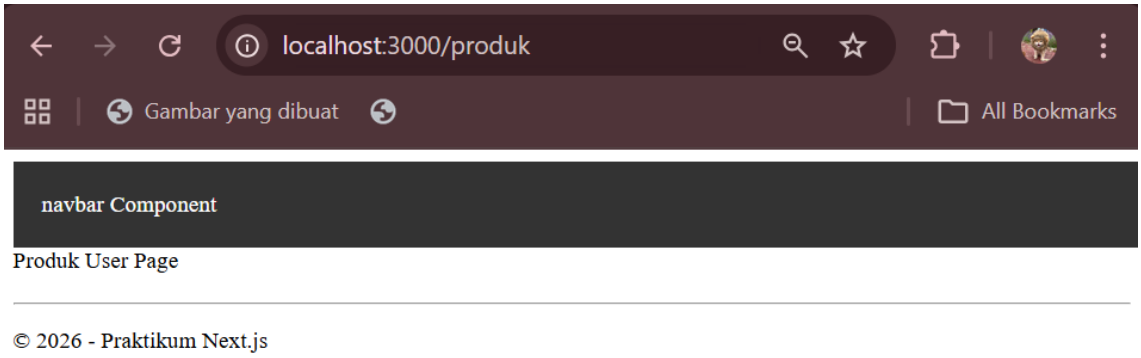
1. Tambahkan button login:

```
import Link from "next/link";
import { useRouter } from "next/router";

const halamanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // logic login disini
        push("/produk");
    }
    return (
        <div>

            <h1>Halaman Login</h1>
            <button onClick={handleLogin}>Login</button> <br />
            <button onClick={() => push('/produk')}>Login</button> <br />
            <button onClick={() => handlerLogin()}>Login</button> <br />
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default halamanLogin;
```

Saat ini gunakan onClick={() => handlerLogin()}

2. Klik tombol dan perhatikan perpindahan halaman tanpa reload.
3. Jika di klik button login maka akan menuju /produk






Note :

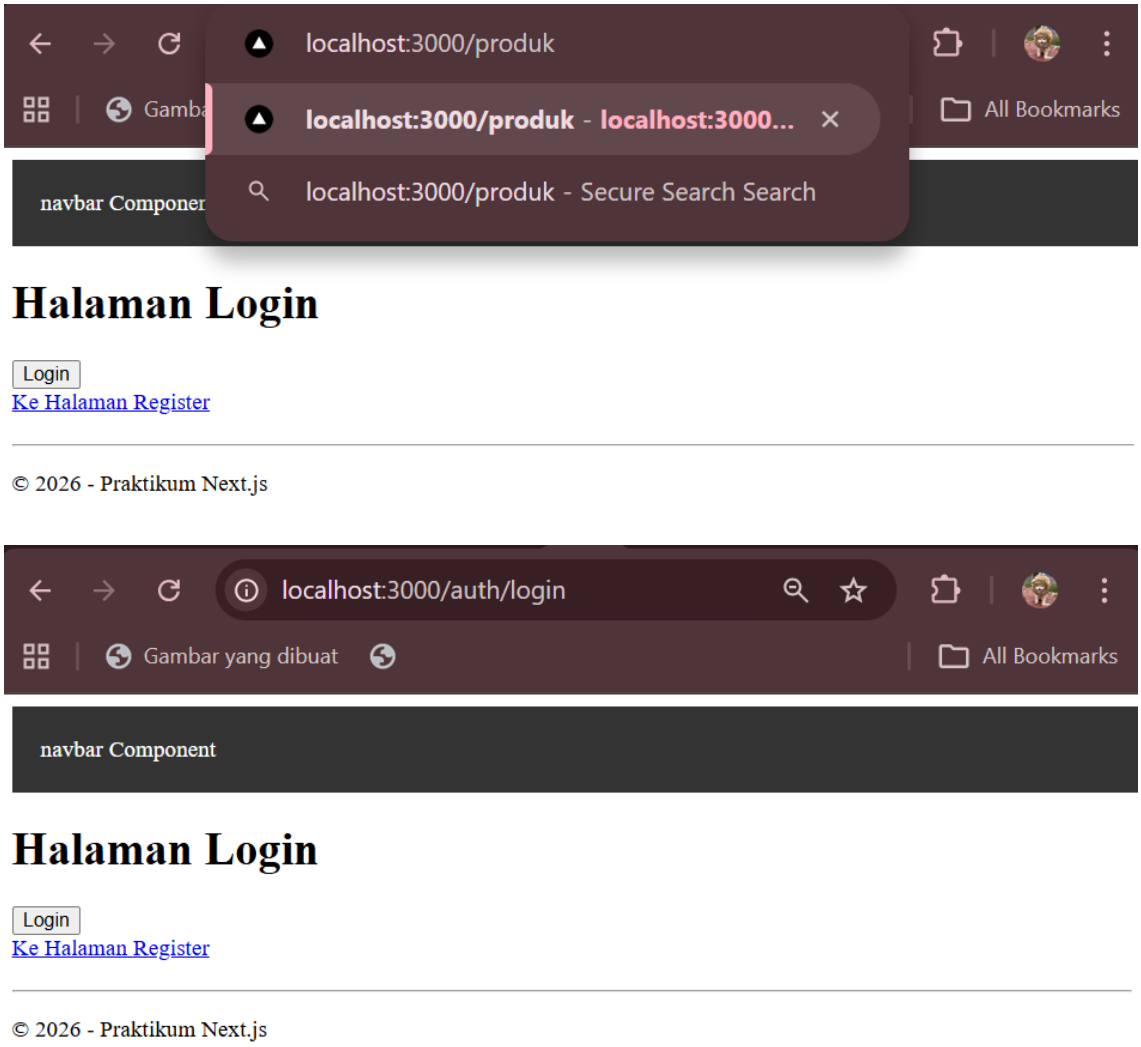
Pastikan code untuk redirect tidak aktif , jika aktif maka ketika masuk ke produk akan lansung redirect ke login

## Langkah 8 – Simulasi Redirect (Belum Login)

1. Di halaman product, pada index.tsx tambahkan beberapa code berikut:

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const produk = () => {
    const [isLogin, setIsLogin] = useState(false);
    const { push } = useRouter();

    useEffect(() => {
        if (!isLogin) {
            push("/auth/login");
        }
    }, []);
    return (
        <div>
            Produk User Page
        </div>
    );
};

export default produk;
```

2. Jika Akses /product → otomatis diarahkan ke login.



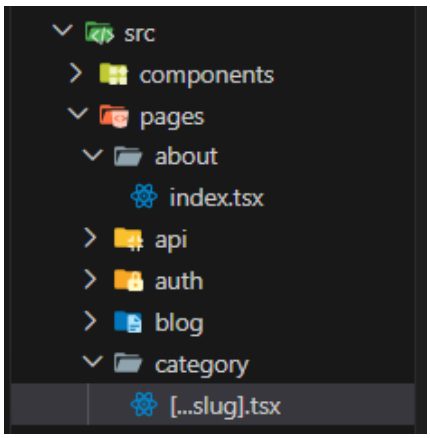




## E. Tugas Praktikum

### Tugas 1 (Wajib)

- Buat catch-all route:
- /category/[...slug].js

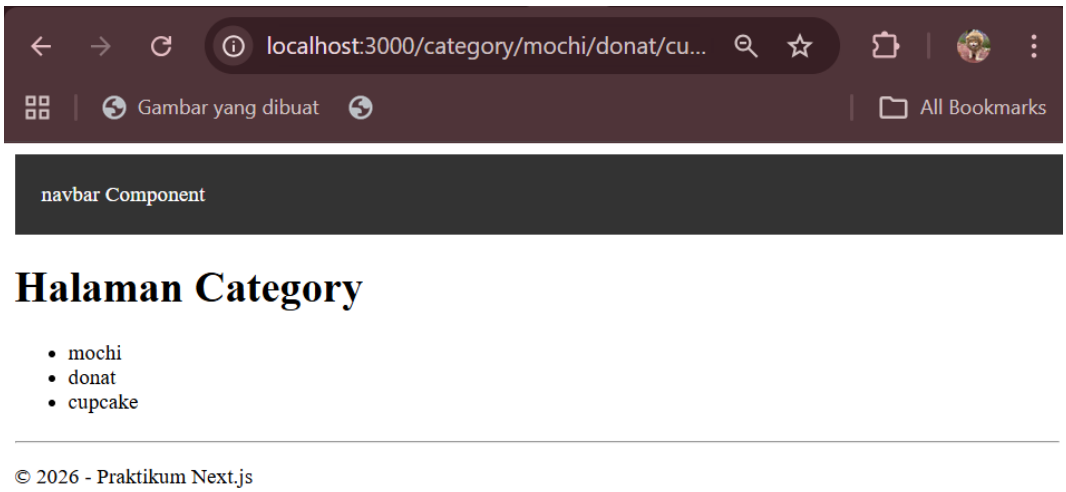


- Tampilkan seluruh parameter URL dalam bentuk list.

Code:

```
import { useRouter } from "next/router";

const CategoryPage = () => {
  const { query } = useRouter();
  const slug = query.slug as string[] | undefined;

  return (
    <div>
      <h1>Halaman Category</h1>

      {Array.isArray(slug) ? (
        <ul>
          {slug.map((item, index) => (
            <li key={index}>{item}</li>
          ))}
        </ul>
      ) : (
        <p>Tidak ada parameter</p>
      )}
    </div>
  );
};

export default CategoryPage;
```

Output:




**Tugas 2 (Wajib)**

- Buat navigasi:
  - Login → Product (imperatif)
  - Login ↔ Register (Link)

```
import Link from "next/link";
import { useRouter } from "next/router";

const halamanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // logic login disini
        push("/produk");
    }
    return (
        <div>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default halamanLogin;
```

Navigasi imperatif digunakan untuk perpindahan halaman melalui fungsi (biasanya setelah proses seperti login), sedangkan navigasi deklaratif digunakan untuk navigasi biasa antar halaman.

**Tugas 3 (Pengayaan)**

- Terapkan redirect otomatis ke login jika user belum login.

Folder auth:

login.jsx

```
import Link from "next/link";
import { useRouter } from "next/router";

const halamanLogin = () => {
    const {push} = useRouter();
    const handleLogin = () => {
        // Simpan status login
        localStorage.setItem("isLogin", "true");

        // logic login disini
        push("/produk");
    }
    return (
        <div>

            <h1>Halaman Login</h1>

            {/* <button onClick={handleLogin}>Login</button> <br /> */}
            {/* <button onClick={() => push('/produk')}>Login</button> <br /> */}

            {/* Navigasi Imperatif */}
            <button onClick={() => handleLogin()}>Login</button> <br />

            {/* Navigasi Link */}
            <Link href="/auth/register">Ke Halaman Register</Link>
        </div>
    );
};

export default halamanLogin;
```

Folder produk:

index.tsx

```tsx
import { useRouter } from "next/router";
import { useEffect } from "react";

const produk = () => {

    const { push } = useRouter();

    // Cek status login saat halaman dibuka
    useEffect(() => {
        const loginStatus = localStorage.getItem("isLogin");

        if (loginStatus !== "true") {
            push("/auth/login");
        }
    }, []);

    return (
        <div>
            Produk User Page
        </div>
    );
};

export default produk;
```

## F. Pertanyaan Evaluasi

1. Apa perbedaan [id].js dan [...slug].js?
   - Pada Next.js, file dengan nama [id].js digunakan untuk membuat dynamic route yang hanya menangkap satu parameter dari URL. Artinya, file ini hanya menerima satu nilai setelah nama folder. Contohnya pada URL /produk/10, maka nilai id yang diterima adalah 10. Dynamic route jenis ini biasanya digunakan untuk menampilkan detail data berdasarkan satu parameter, seperti detail produk atau detail user.

     Sedangkan [...slug].js merupakan catch-all route yang dapat menangkap lebih dari satu parameter dalam URL. Semua segmen setelah nama folder akan ditangkap menjadi satu variabel. Contohnya pada URL /category/makanan/minuman/dingin, maka seluruh bagian setelah /category/ akan ditangkap dalam satu variabel bernama slug. Oleh karena itu, [...slug].js lebih fleksibel karena dapat menangani URL dengan banyak tingkatan.

2. Mengapa slug berbentuk array?
   - Slug berbentuk array karena file [...slug].js dapat menangkap lebih dari satu segmen URL. Setiap bagian URL yang dipisahkan oleh tanda garis miring (/) akan dimasukkan sebagai elemen dalam array.

     Sebagai contoh, jika URL yang diakses adalah /category/elektronik/laptop/gaming, maka nilai slug akan menjadi ["elektronik", "laptop", "gaming"]. Walaupun hanya terdapat satu segmen, misalnya

/category/makanan, nilai slug tetap berbentuk array yaitu ["makanan"]. Hal ini dilakukan agar struktur datanya konsisten dan dapat digunakan untuk berbagai kebutuhan, seperti kategori bertingkat.

3. Kapan sebaiknya menggunakan Link dan router.push()?
   - Link biasanya digunakan untuk navigasi biasa yang dilakukan melalui klik pada teks atau tombol yang berfungsi seperti menu. Contohnya berpindah dari halaman Login ke Register atau dari halaman utama ke halaman About. Cara ini disebut navigasi deklaratif karena tujuan navigasi sudah ditentukan langsung pada komponen tersebut.

     Sedangkan router.push() digunakan ketika perpindahan halaman terjadi karena suatu aksi atau kondisi tertentu. Contohnya setelah user berhasil login, setelah submit form, atau ketika sistem melakukan redirect otomatis jika user belum login. Cara ini disebut navigasi imperatif karena perpindahan halaman dilakukan melalui perintah di dalam fungsi.

4. Mengapa navigasi Next.js tidak me-refresh halaman?
   - Navigasi pada Next.js tidak me-refresh seluruh halaman karena menggunakan konsep client-side routing. Artinya, ketika user berpindah halaman, Next.js hanya mengganti bagian konten yang diperlukan tanpa memuat ulang seluruh halaman dari awal.

     Hal ini membuat perpindahan halaman terasa lebih cepat dan lebih responsif dibandingkan website tradisional yang selalu melakukan reload penuh. Selain itu, pengalaman pengguna menjadi lebih baik karena tidak terjadi kedipan layar (flicker) dan beberapa state pada aplikasi dapat tetap dipertahankan.